=)

Estructuras a definir:

- Una clase **Usuario**: un objeto con los atributos ID (int), millas disponibles (double), millas totales (double), millasCanjeadasEnviadas (double), transferencias (Conj<int>), nodoEnABB (nodoABB), nodoEnHeap (NodoHeap).
- Un diccionario implementado por medio de un **ABB**, las claves de los nodos serían el ID del usuario y el valor el objeto usuario que definimos arriba.
- Un **Heap con Handles** respecto a las millas disponibles del usuario que representa el ranking.
- Un **Trie** para ambos tipos de promociones, el último nodo de cada promoción será un objeto que almacena el factor, y en caso de tener, el tope.

Atributos del sistema:

- ABB.
- Heap.
- elMasAcumulador (int).
- elMasCanjeador (int).

Justificaciones:

- Unificamos las millas canjeadas con las enviadas porque es el cálculo que plantea el enunciado del TP1 (puede cambiar en un futuro al hablarlo con el equipo de trabajo). Por otro lado, en transferencias tenemos un conjunto, no tiene repetidos, y los elementos del conjunto el id con el cual se hizo la transacción. Luego para saber si son exclusivos, se verifica si la longitud del conjunto es igual a uno, de esta manera obtenemos O(U) en paresExclusivosTransferidores.
- Definimos implementar un diccionario como un ABB para cumplir con la complejidad solicitada en las operaciones que requieren una búsqueda de usuarios. Por lo tanto, su función principal es buscar los a los usuarios del sistema respecto de su ID. Por otro lado elegimos definir un Heap con Handles centrado en las millas disponibles para ordenar a los usuarios respecto de sus millas disponibles y para tener una complejidad O(N log U) (siendo N la cantidad de elementos de la respuesta y log U la búsqueda de los elementos dentro del Heap con Handle) en la operación rankingMillas. Por último definimos un Trie para las promociones porque es la estructura que tiene una complejidad lineal relacionada con la longitud de una palabra y no con la cantidad de promociones en si, de esta manera se cumple la complejidad en las operaciones iniciarPromoción, iniciarPromociónConTope, finalizarPromoción, finalizarPromociónConTope.
- La conexión intermedia entre estructuras (ABB y Heap con Handle) es la clase Usuario. En cada instancia vamos a definir los atributos para conectar ambas estructuras y que eso nos permita una vinculación con complejidad O(1) entre las estructuras, obtenida gracias a que guardamos el puntero al nodo de cada estructura en cada instancia del objeto Usuario.
- Para este sistema suponemos que tenemos carga de datos aleatoria, es decir una distribución uniforme de las claves y de esa forma la altura del ABB es O(log U) y por lo tanto las operaciones de búsqueda tienen complejidad O(log U). Ahora en el peor caso, claves ordenadas, la complejidad es O(U), la misma que pide cada uno de los enunciados que requiere búsqueda entre usuarios.

- Tanto elMasCanjeador como elMasAcumulador tienen complejidad de O(1) porque consultan atributos del sistema. Esto es posible gracias a que se actualizan los valores en las operaciones correspondientes: acumularMillas y registrarUsuarioConReferidor sobre elMasAcumulador; canjearMillas y transferirMillas sobre elMasCanjeador. Al inicio de la ejecución estos atributos valen cero.

Método público en Usuario:

- factorCategoria: a partir de las millas totales del usuario, calcula el factor de categoría que le corresponde y lo devuelve.

Para guiarnos en el trabajo:

- consultarSaldo: busca el id (ABB) y devuelve las disponibles. No usamos el Heap.
- acumularMillas: busca el id (ABB), actualiza las disponibles y totales y se re-acomoda el Heap. De ser necesario hay que actualizar el valor de elMasAcumulador.
- canjearMillas: busca el id (ABB), actualiza las disponibles sin tocar las totales y re-acomoda el Heap. De ser necesario hay que actualizar el valor de elMasCanjeador.
- transferirMillas: busca el id (ABB), actualiza las millas disponibles de cada uno y re-acomoda el Heap. De ser necesario hay que actualizar el valor de elMasCanjeador.
- consultarCategoria: busca el id (ABB) y obtiene la información del objeto Usuario con el id que le envían (cálculo definido en el TP1).
- registrarUsuarioConReferidor: agrega el nuevo usuario en ambas estructuras y re-acomoda el heap. De ser necesario hay que actualizar el valor de elMasAcumulador.
- rankingMillas: solo copia los id, no altera el Heap con Handle.
- El ABB es una suerte de búsqueda principal y el Heap sería un orden secundario que nos sirve para la operación rankingMillas.
- El atributo elMasAcumulador se tiene que actualizar en las operaciones que modifican las millas totales (acumularMillas y registrarUsuarioConReferidor).
- El atributo elMasCanjeador se tiene que actualizar en las operaciones que modifica millasCanjeadasEnviadas (canjearMillas y transferirMillas).

Para pensar o debatir / pendiente:

- paresTransferidoresExclusivos: pensar si transferencias y la salida como un O(U) cumplen. Yo creo que sí porque recorres todos los usuarios O(U) y después la comparación |transferencias| == 1 y agregar el par al conjunto de salida son los dos O(1). Analizar pares duplicados.
- Si se ingresa un usuario y luego uno nuevo con el mismo ID ¿Cómo lo solucionamos?
  Rta: lanzar error.
- El heap se va a implementar sobre un array y por lo tanto el nodoHeap (atributo) que representa al Handle, guarda la posición del array donde está el nodo del Heap. Además cuando haya sift up/down también hay que cambiarlo en el array. Pensar cómo aplicar la fórmula que nos dieron para definir rankingMillas.
  Rta. : métodos privados para sift up/down.

- ¿Hace falta guardar categoría?
  Rta. : no.

(No prestarle tanta atención)

Ideas en el aire:

- Objeto usuario con atributos: ID, millasD, millasT, categoría, exclusivo, …
- Definir un ABB/AVL con claves ID que referencie al objeto para consultarSaldo, acumularMillas, canjearMillas, transferirMillas, consultarCategoria, registrarUsuarioConRef.
- Definir un Heap con Handles con claves millas (revisar idea) que también referencie a usuario.
- La conexión entre estructuras son justamente los Handles, porque la relación que hace es entre punteros (desde ABB hacia Heap o viceversa) entonces O(1). Idea sacada de estructuras_enlazadas.pdf
- Debatir cómo se mide elMasCanjeador.
- transferirMillas tiene que verificar que el usuario siga siendo exclusivo.
- Tries para las promociones, en el último nodo guardamos un objeto promoción con los atributos necesarios llegar hasta el final del nombre de la promoción es O(|L|) y acceder a los atributos del objeto es O(1). De esta manera unificamos las promociones en una sola estructura y la diferencia entre promociones se da en el último nodo de cada promoción. Sobre la complejidad del Trie no estoy muy seguro, lo estoy intuyendo porque no hay información sobre su complejidad.

Para el segundo punto podemos definir tanto un ABB como un AVL, ambos respetan la complejidad pedida pero hay que tener en cuenta dos cosas:

- ABB: cumple el peor caso y el caso promedio sií hay una distribución uniforme de las claves.
- AVL: tanto en el caso promedio como en el peor son O(log N), en este caso no me termina de quedar claro si necesariamente tenemos que llegar a O(N), la complejidad de todas las operaciones del primer punto, o implementando un algoritmo de menor complejidad seguimos cumpliendo. Intuitivamente tiene sentido que si haces un algoritmo con menor complejidad cumplas la cota superior, pero no estoy seguro.

Cualquiera sea la decisión todo esto debemos explicarlo en justificacion.pdf.